

Application-specific access policies using Zscaler Private Access (ZPA).

List of Teammates

S. NO	NAME	COLLEGE	EMAIL - ID
1.	Mathiyarasu S V	VETIAS, Thindal	mathiyarasusv23cse@vetias.ac.in
2.	Thirumugan A P	VETIAS, Thindal	thirumuganap23cse@vetias.ac.in
3.	Barat Raj R R	VETIAS, Thindal	baratrajrr23cse@vetias.ac.in
4.	Karthick T	VETIAS, Thindal	karthickt23cse@vetias.ac.in
5.	Sandhiya S	VETIAS, Thindal	sandhiyas23cse@vetias.ac.in

Title of the Project: -

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Overview: -

In an era where the digital landscape is constantly evolving, organizations face an ever-increasing threat from cyberattacks. The need for swift and effective incident response has become paramount to safeguard sensitive data, protect digital assets, and maintain the trust of stakeholders. In response to this growing challenge, we present a visionary project: the development of an AI-driven incident response platform that empowers cybersecurity teams with the tools to automate and enhance incident triage and response tasks.

In response to the evolving threat landscape and the dissolution of the traditional network perimeter, modern enterprises are rapidly shifting away from legacy VPN-centric remote access models. These conventional approaches often grant excessive network-level access, creating a significant attack surface and increasing the risk of lateral threat movement. To address these challenges, this project centers on the implementation of a Zero Trust security architecture, fundamentally operating on the principle of "never trust, always verify." By moving beyond location-based access, we aim to establish a security posture where trust is no longer implicit, and access is determined by granular context rather than network connectivity.

The core of this initiative involves the deployment of Zscaler Private Access (ZPA), a cloud-delivered Zero Trust Access solution designed to create identity-aware, application-specific connectivity. Unlike traditional VPNs that place users on the internal network, ZPA leverages a "service-initiated" model to establish direct, one-to-one connections between authorized users and specific applications, completely hiding the corporate network and IP address space from the internet. This ensures that users connect exclusively to the applications they are permitted to access, based on continuous identity verification and context, rendering the applications invisible to unauthorized users and preventing common attack vectors such as port scanning and distributed denial-of-service (DDoS) attacks.

Ultimately, this project will enforce a least-privilege access model, ensuring users and devices are granted the minimum required access necessary to perform their functions. By decoupling application access from network access, we will eliminate the risk of lateral movement across the network, as users have no visibility or connectivity to other hosts. This strategic move not only significantly enhances security by reducing the attack surface but also simplifies the user experience and reduces operational overhead associated with legacy remote access infrastructure, positioning the organization at the forefront of secure and efficient access management.

Abstract: -

The proliferation of hybrid work and cloud-based resources has rendered traditional perimeter-based security models, such as VPNs, obsolete and increasingly dangerous. These legacy solutions often provide overly broad network access, exposing organizations to significant risks including lateral threat movement and an expanded attack surface. This project addresses these vulnerabilities by implementing a Zero Trust Architecture (ZTA) utilizing Zscaler Private Access (ZPA). The primary objective is to transition from network-centric to identity-centric access, thereby enforcing the principle of least privilege.

The methodology involves deploying ZPA's cloud-based service to create application-specific, identity-aware micro-tunnels. Unlike conventional VPNs that place users on the internal network, this solution establishes direct, one-to-one connections between verified users and authorized applications. The network infrastructure and all non-authorized applications remain completely invisible to the end-user, effectively eliminating surface for reconnaissance and attack. Access policies are dynamically governed by user identity, device posture, and contextual attributes, ensuring that connectivity is strictly limited to what is explicitly required for specific job functions.

The implementation of ZPA successfully decouples application access from network access, resulting in a demonstrable reduction in the organization's attack surface and the complete prevention of lateral movement. This project delivers a robust, scalable, and user-friendly remote access framework that aligns with modern security best practices. By shifting to an identity-aware, least-privilege model, the organization significantly enhances its security posture while simultaneously reducing operational complexity and improving the end-user experience.

Use Case Scenarios: -

Scenario 1: Finance Department Access to ERP System A senior accountant in the finance department needs to access the corporate Enterprise Resource Planning (ERP) system to process monthly payroll. The user connects from a remote location using their company-managed laptop. Upon launching the ZPA Client Connector, they authenticate via Azure Active Directory with multi-factor authentication (MFA). ZPA evaluates the user's identity, group membership (Finance_ERP_Users), device posture (antivirus active, disk encryption enabled), and connection context. The policy engine determines that the user is authorized and brokers a secure, direct micro-tunnel to the ERP application only. The user can interact with the ERP interface, but has no visibility or network-level access to any other internal systems, such as development servers or the HR database.

Output:

- User authenticated and authorized based on identity and group membership
- Direct micro-tunnel established specifically to the ERP application
- All other internal applications and network segments remain invisible and inaccessible

Scenario 2: Contractor Access to Specific Project Repository An external software consultant is hired for a three-month project and requires access only to a specific Git repository hosted on an internal server. The contractor is onboarded as a guest user in the identity provider and assigned to the "Project_X_Contractors" group. When the contractor connects using the ZPA client, the system verifies their identity and checks the device for basic security compliance (OS patch level, firewall enabled). ZPA enforces a policy that grants access exclusively to the Git repository application (port 22/SSH or port 443/HTTPS) during business hours (9 AM to 5 PM). The contractor can clone, push, and pull code, but cannot discover or connect to any other internal resources, including the corporate intranet or file servers.

Output:

- Time-bound and role-specific access enforced
- Contractor restricted to a single application (Git repository)
- Lateral movement to adjacent systems completely blocked

Scenario 3: Executive Access to Confidential Board Documents A company executive needs to access a confidential folder containing board meeting materials hosted on an internal file server. The executive connects from a tablet device while traveling internationally. ZPA integrates with the mobile device management (MDM) solution to verify that the tablet is compliant (not jailbroken, passcode enabled, encryption active). Upon successful authentication, ZPA evaluates the policy: "If user is in 'Executive_Board' group AND device is MDM-compliant AND connection is from a trusted country (allowlist), THEN grant access to 'Board_Documents' application." The executive is connected directly to the specific file share, and all data in transit is encrypted via TLS. Any attempt to access other shares or network resources is automatically blocked and logged.

Output:

- Context-aware access based on user role, device posture, and geographic location
- Granular access to a specific file share (Board_Documents)
- Comprehensive logging of access events for audit and compliance

Scenario 4: IT Administrator Access to Server Management Interfaces

A systems administrator requires access to management interfaces (SSH for Linux servers and RDP for Windows servers) to perform routine maintenance. Instead of granting VPN access to the entire management network, ZPA is configured to treat each server or management interface as a separate application segment. The administrator authenticates and is presented only with the specific servers they are authorized to manage based on their IT subgroup (e.g., "Linux_Admins" see only Linux servers; "Windows_Admins" see only Windows servers). When the admin connects via SSH to a Linux server, ZPA creates a direct micro-tunnel to that specific IP and port combination. The admin cannot scan the subnet, access other servers, or initiate connections to non-authorized ports.

Output:

- Micro-segmentation at the individual server/interface level
- Role-based access controlling visibility of specific management consoles
- Elimination of network-level reconnaissance capabilities for admins

Scenario 5: Mergers and Acquisitions Integration A company acquires a smaller firm and needs to provide the acquired company's HR team with temporary access to the parent company's benefits enrollment application. The acquired company's users are added as

external collaborators in the identity provider with limited attributes. ZPA policies are created that specifically grant the "Acquired_HR" group access only to the "Benefits_App" (a web application hosted internally). The acquired users install the ZPA client, authenticate, and are connected directly to the benefits application. Because ZPA provides access without network connectivity, the parent company's entire internal network remains completely hidden and protected. The acquired users cannot ping, scan, or access any other corporate resource. After six months, when the integration is complete and users are migrated to the parent identity system, the temporary policies are revoked.

Output:

- Secure, temporary access provided without network exposure
- Complete network invisibility protecting the parent company's infrastructure
- Simplified deprovisioning through policy revocation rather than complex network change

Stage – 1

Title of the Project: -

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

List of Vulnerability Table: -

S.No.	Vulnerability Name	CWE/CVE - No
1.	Absolute PathTraversal	CWE-36
2.	Password in Configuration File	CWE-260
3.	Use of Obsolete Function	CWE-447
4.	Insufficient Entropy	CWE-331
5.	Missing XML Validation	CWE-112
6.	Incorrect Privilege Assignment	CWE-266
7.	Authentication Bypass by Alternate Name	CWE-289
8.	Deserialization of Untrusted Data	CWE-502
9.	Information Loss of Omission	CWE-221
10.	NULL Pointer Dereference	CWE-476

REPORT: -

1. Vulnerability Name: - Absolute Path Traversal

CWE/CVE: - CWE-36

OWASP/SANS Category: - A01:2025 Broken Access Control

Description: -

Absolute Path Traversal is a type of web security vulnerability that occurs when an application allows a user to access files on a server using an absolute file path without proper validation or access control. An absolute path specifies the full location of a file starting from the root directory of the operating system.

In this vulnerability, an attacker can manipulate input parameters (such as file names or paths) to access sensitive files or directories on the server that should normally be restricted. Instead of limiting users to specific directories, the application mistakenly allows direct access to files using complete system paths.

Business Impact:-

Absolute Path Traversal can have serious impacts on a business if attackers exploit the vulnerability. It may allow unauthorized access to sensitive files such as configuration files, user data, and system information. This can lead to data breaches and exposure of confidential business information. Organizations may suffer financial losses due to recovery costs, legal penalties, and compensation. Such incidents can also damage the company's reputation and reduce customer trust. Attackers may use the obtained information to perform further cyberattacks on the system. Overall, this vulnerability can affect business operations, security, and long-term reliability.

CWE-36: Absolute Path Traversal

Weakness ID: 36
Vulnerability Mapping: **ALLOWED**
Abstraction: Base

View customized information:

Description
The product uses external input to construct a pathname that should be within a restricted directory, but it does not properly neutralize absolute path sequences such as "/abs/path" that can resolve to a location that is outside of that directory.

Extended Description
This allows attackers to traverse the file system to access files or directories that are outside of the restricted directory.

Potential Mitigations

Phase(s)	Mitigation
Implementation	Strategy: Input Validation Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if the input is only expected to contain colors such as "red" or "blue." Do not rely exclusively on looking for malicious or malformed inputs. This is likely to miss at least one undesirable input, especially if the code's environment changes. This can give attackers enough room to bypass the intended validation. However, denylists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When validating filenames, use stringent allowlists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses such as CWE-23, and exclude directory separators such as "/" to avoid CWE-36. Use a list of allowable file extensions, which will help to avoid CWE-434.

2. Vulnerability Name: - Password in Configuration File

CWE/CVE: - CWE-260

OWASP/SANS Category: - A02:2025 Security Misconfiguration

Description: -

Password in Configuration File is a security vulnerability where passwords are stored directly in application configuration files in plain text. These files are often used to store system settings, database credentials, and authentication details. If the configuration file is accessed by unauthorized users, they can easily view the stored passwords. Attackers may obtain these files through system access, backups, or other vulnerabilities. Storing passwords in plain text increases the risk of credential theft and unauthorized access. This issue usually occurs due to poor security practices during application development. Using encryption or secure credential management systems can help prevent this vulnerability.

Business Impact:-

Storing passwords in configuration files can expose sensitive credentials if the files are accessed by attackers. This can allow unauthorized

users to gain access to databases, servers, or applications. As a result, confidential business and customer data may be compromised. Organizations may experience financial losses due to security breaches and system recovery costs. It can also damage the company's reputation and reduce customer confidence. Businesses may face legal and compliance issues for failing to protect sensitive information. Overall, this vulnerability can lead to serious security and operational risks.

The screenshot shows the CWE-260: Password in Configuration File page. The page header includes the CWE logo, the title 'Common Weakness Enumeration', and a tagline 'A community-developed list of SW & HW weaknesses that can become vulnerabilities'. There are also badges for 'Top 25' and 'Top HW CWE', and a 'New to CWE? Start here!' link. The main content area is titled 'CWE-260: Password in Configuration File' and includes sections for 'Description', 'Extended Description', 'Potential Mitigations', and 'Relationships'. The 'Description' section states: 'The product stores a password in a configuration file that might be accessible to actors who do not know the password.' The 'Extended Description' section states: 'This can result in compromise of the system for which the password is used. An attacker could gain access to this file and learn the stored password or worse yet, change the password to one of their choosing.' The 'Potential Mitigations' section includes a table with two rows: 'Architecture and Design' with the mitigation 'Avoid storing passwords in easily accessible locations.' and 'Architecture and Design' with the mitigation 'Consider storing cryptographic hashes of passwords as an alternative to storing in plaintext.' The 'Relationships' section includes a table with four columns: 'Nature', 'Type', 'ID', and 'Name'. The table lists four relationships: 'ChildOf' (ID 522, Name 'Insufficiently Protected Credentials'), 'ParentOf' (ID 13, Name 'ASP.NET Misconfiguration: Password in Configuration File'), 'ParentOf' (ID 258, Name 'Empty Password in Configuration File'), and 'ParentOf' (ID 555, Name '12FF Misconfiguration: Plaintext Password in Configuration File').

3. VulnerabilityName:- Use of Obsolete Function

CWE/CVE: CWE-447

OWASP/SANS Category:- A03:2025 Software Supply Chain Failures

Description: -

Use of Obsolete Function is a security vulnerability that occurs when an application uses outdated or deprecated functions in its code. These functions are often no longer recommended because they may contain security weaknesses. Developers may continue using them due to legacy code or lack of updates. Obsolete functions may not follow modern security standards. Attackers can exploit weaknesses in these functions to compromise the system. This vulnerability is common in older software or poorly maintained applications. Regular code updates and using secure alternatives can help prevent this issue.

Business Impact:- Using obsolete functions can expose applications to known security vulnerabilities. Attackers may exploit these weaknesses to gain unauthorized access to systems or data. This can lead to data breaches and system compromise. Organizations may face financial losses due to security incidents and system repairs. It can also damage the company's reputation and reduce customer trust. Businesses may experience operational disruptions if the system becomes unstable or compromised. Keeping software updated helps reduce these security risks.

The screenshot shows the CWE-447: Unimplemented or Unsupported Feature in UI page. The page header includes the CWE logo, the title 'Common Weakness Enumeration', and a tagline 'A community-developed list of SW & HW weaknesses that can become vulnerabilities'. There are also badges for 'Top 25' and 'HW CWE', and a 'New to CWE? Start here!' link. The breadcrumb trail is 'Home > CWE List > CWE-447: Unimplemented or Unsupported Feature in UI (4.19.1)'. The page has a navigation bar with links: Home, About, Learn, Access Content, Community, and Search. The main content area is titled 'CWE-447: Unimplemented or Unsupported Feature in UI'. It shows the Weakness ID: 447, Vulnerability Mapping: ALLOWED, and Abstraction: Base. There are buttons for 'View customized information:' with options: Conceptual, Operational, Mapping Friendly, Complete, and Custom. The 'Description' section states: 'A UI function for a security feature appears to be supported and gives feedback to the user that suggests that it is supported, but the underlying functionality is not implemented.' The 'Potential Mitigations' section has a table with one entry: 'Testing' with the mitigation 'Perform functionality testing before deploying the application.' The 'Relationships' section has two views: 'Relevant to the view "Research Concepts" (View-1000)' and 'Relevant to the view "Software Development" (View-699)'. The first view shows two entries: 'ChildOf' with ID 446 (UI Discrepancy for Security Feature) and ID 671 (Lack of Administrator Control over Security). The second view shows one entry: 'MemberOf' with ID 355 (User Interface Security Issues). The 'Modes Of Introduction' section is partially visible at the bottom.

4. Vulnerability Name: - Insufficient Entropy

CWE/CVE: CWE-331

OWASP/SANS Category: - A04:2025 Cryptographic Failures

Description: -

Insufficient Entropy is a security vulnerability that occurs when an application generates predictable random values. Entropy refers to the level of randomness used in generating data such as passwords, session IDs, or encryption keys. If the randomness is weak or predictable, attackers may be able to guess or reproduce these values. This can happen when insecure random number generators are used. As a result, attackers may hijack sessions or break authentication mechanisms..

Business Impact:-

Insufficient entropy can allow attackers to predict sensitive values such as session IDs or authentication tokens. This may lead to unauthorized access to user accounts or system resources. Attackers may also bypass security controls or encryption mechanisms. Organizations may experience data breaches and exposure of confidential information. This can result in financial losses due to recovery and security improvements. It may also damage the company's reputation and reduce customer trust. Proper security practices are necessary to protect business systems from such risks.

The screenshot shows the CWE-331: Insufficient Entropy page. It includes the CWE logo, a navigation bar with links like Home, About, Learn, Access Content, Community, and Search. The main content area displays the weakness ID (331), vulnerability mapping (ALLOWED), and abstraction (Base). It also features a description, potential mitigations, and a table of relationships. The relationships table is divided into two sections: 'Relevant to the view "Research Concepts" (View-1000)' and 'Relevant to the view "Software Development" (View-699)'. The first section lists three relationships: ChildOf (330), ParentOf (332), and ParentOf (333). The second section lists two relationships: MemberOf (310) and MemberOf (1213).

Nature	Type	ID	Name
ChildOf	Green circle	330	Use of Insufficiently Random Values
ParentOf	Purple circle	332	Insufficient Entropy in PRNG
ParentOf	Purple circle	333	Improper Handling of Insufficient Entropy in TRNG

Nature	Type	ID	Name
MemberOf	Red square	310	Cryptographic Issues
MemberOf	Red square	1213	Random Number Issues

5. Vulnerability Name: - Missing XML Validation

CWE/CVE: OWASP WASC SANS Category: - Description: -

Missing XML Validation is a security vulnerability that occurs when an application processes XML input without properly validating its structure and content. XML validation ensures that the data follows a defined format and schema. Without validation, attackers can send malicious or unexpected XML data to the system. This can lead to issues such as XML injection or XML External Entity (XXE) attacks. Improperly handled XML input may allow

attackers to access sensitive files or disrupt application behavior. The vulnerability often occurs in applications that rely heavily on XML-based communication. Implementing proper XML schema validation and secure parsing helps prevent this issue.

Business Impact:-

Missing XML validation can allow attackers to send malicious XML data to the application. This may lead to unauthorized access to sensitive files or system information. Attackers could exploit the vulnerability to perform further attacks on the system. Organizations may experience data breaches and exposure of confidential information. This can result in financial losses due to recovery costs and legal penalties. It may also damage the organization's reputation and reduce customer trust. Proper input validation is important to protect business systems from such threats.

The screenshot shows the CWE-112: Missing XML Validation page. The header includes the CWE logo, navigation links (Home, About, Learn, Access Content, Community, Search), and a search bar. The main content area displays the weakness ID (112), vulnerability mapping (ALLOWED), and abstraction (Base). It includes tabs for View customized information (Conceptual, Operational, Mapping Friendly, Complete, Custom). The Description section states: 'The product accepts XML from an untrusted source but does not validate the XML against the proper schema.' The Extended Description section explains: 'Most successful attacks begin with a violation of the programmer's assumptions. By accepting an XML document without validating it against a DTD or XML schema, the programmer leaves a door open for attackers to provide unexpected, unreasonable, or malicious input.' The Potential Mitigations section lists a phase (Architecture and Design) with a mitigation strategy: 'Input Validation'. The Relationships section shows a table with columns: Nature, Type, ID, Name. The table contains one entry: 'ChildOf', 'U', '1286', 'Improper Validation of Syntactic Correctness of Input'.

Nature	Type	ID	Name
ChildOf	U	1286	Improper Validation of Syntactic Correctness of Input

6. VulnerabilityName:- Incorrect Privilege Assignment

CWE/CVE: CWE-266

OWASP/SANS Category: - A06:2025 Insecure Design

Description: -

Incorrect Privilege Assignment is a security vulnerability that occurs when users or processes are given more permissions than necessary. Access privileges should be limited based on the user's role and responsibilities. When

excessive privileges are assigned, unauthorized actions may become possible. For example, a normal user might gain administrative rights or access to restricted data. This usually happens due to poor access control policies or configuration errors. Attackers can exploit these privileges to modify, delete, or access sensitive information. Implementing the principle of least privilege helps prevent this vulnerability.

Business Impact:-

Incorrect privilege assignment can allow unauthorized users to access sensitive systems or data. Attackers may misuse elevated privileges to modify or steal important information. This can lead to data breaches and system compromise. Organizations may suffer financial losses due to security incidents and recovery costs. Such incidents can damage the company's reputation and reduce customer trust. Businesses may also face legal or compliance issues for failing to enforce proper access control. Proper privilege management is essential to maintain system security and protect business operations.

The screenshot shows the CWE-266: Incorrect Privilege Assignment page. The page header includes the CWE logo and navigation links. The main content area displays the weakness ID (266), vulnerability mapping (ALLOWED), and abstraction (Base). It provides a description: 'A product incorrectly assigns a privilege to a particular actor, creating an unintended sphere of control for that actor.' Under 'Common Consequences', it lists 'Impact' (Gain Privileges or Assume Identity) and 'Details' (Scope: Access Control). Under 'Potential Mitigations', it lists 'Phase(s)' (Architecture and Design; Operation) and 'Mitigation' (Very carefully manage the setting, management, and handling of privileges. Explicitly manage trust zones in the software. Strategy: Environment Hardening).

7. VulnerabilityName:- Authentication Bypass by Alternate Name

CWE/CVE: CWE-289

OWASP/SANS Category: - A07:2025 Authentication Failures

Description: -

Authentication Bypass by Alternate Name is a security vulnerability where an attacker gains access to a protected resource using an alternative name

or path. This happens when an application checks authentication only for specific file names or URLs. Attackers may use different representations of the same resource to bypass security checks. For example, accessing a file through an alternate path or encoded URL may avoid authentication controls. This occurs due to improper access validation in the application. As a result, attackers may gain unauthorized access to restricted pages or data. Proper authentication checks and path validation are necessary to prevent this vulnerability.

Business Impact:-

Authentication bypass can allow attackers to access restricted systems or sensitive information without proper login credentials. This may lead to unauthorized data access and system misuse. Confidential business or customer information could be exposed. Organizations may face financial losses due to data breaches and security incidents. Such vulnerabilities can damage the company's reputation and reduce customer trust. Businesses may also face regulatory or compliance penalties. Strong authentication controls are essential to protect business systems and data.

The screenshot shows the CWE-289: Authentication Bypass by Alternate Name page. The page header includes the CWE logo, navigation links (Home, About, Learn, Access Content, Community, Search), and a search bar. The main content area displays the weakness ID (289), vulnerability mapping (ALLOWED), and abstraction (Base). Below this, there are tabs for 'View customized information' (Conceptual, Operational, Mapping Friendly, Complete, Custom). The 'Description' section states: 'The product performs authentication based on the name of a resource being accessed, or the name of the actor performing the access, but it does not properly check all possible names for that resource or actor.' The 'Potential Mitigations' section is a table with two columns: 'Phase(s)' and 'Mitigation'.

Phase(s)	Mitigation
Architecture and Design	Strategy: Input Validation Avoid making decisions based on names of resources (e.g. files) if those resources can have alternate names.
Implementation	Strategy: Input Validation Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if the input is only expected to contain colors such as "red" or "blue." Do not rely exclusively on looking for malicious or malformed inputs. This is likely to miss at least one undesirable input, especially if the code's environment changes. This can give attackers enough room to bypass the intended validation. However, denylists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

8. Vulnerability Name: - Deserialization of Untrusted Data

CWE/CVE: CWE-502

OWASP/SANS Category: - A08:2025 Software or Data Integrity Failures

Description: - Deserialization of Untrusted Data is a security vulnerability that occurs when an application processes serialized data from an untrusted source without proper validation. Serialization converts objects or data into a format that can be stored or transmitted. During deserialization, the data is converted back into objects used by the application. If the data is maliciously modified, it may execute harmful code during the process. Attackers can exploit this to perform unauthorized actions or gain control of the system. This vulnerability often occurs in applications that exchange serialized objects over networks. Proper validation and secure deserialization methods are required to prevent this issue. **Business Impact:-**

Deserialization vulnerabilities can allow attackers to execute malicious code on the system. This may lead to unauthorized access, system compromise, or data manipulation. Sensitive business and customer information could be exposed or modified. Organizations may experience financial losses due to system damage and recovery efforts. It can also disrupt business operations if critical systems are affected.

CWE-502: Deserialization of Untrusted Data

Weakness ID: 502
Vulnerability Mapping: **ALLOWED**
Abstraction: Base

View customized information:

Description

The product deserializes untrusted data without sufficiently ensuring that the resulting data will be valid.

Alternate Terms

Marshaling, Unmarshaling	Marshaling and unmarshaling are effectively synonyms for serialization and deserialization, respectively.
Pickling, Unpickling	In Python, the "pickle" functionality is used to perform serialization and deserialization.
PHP Object Injection	Some PHP application researchers use this term when attacking unsafe use of the unserialize() function; but it is also used for CWE-915 .

Common Consequences

Impact	Details
Modify Application Data; Unexpected	Scope: Integrity Attackers can modify unexpected objects or data that was assumed to be safe from modification. Deserialized data or code could be modified without

9. Vulnerability Name: - Information Loss of Omission

CWE/CVE: CWE-221

OWASP/SANS Category: - A09:2025 Security Logging & Alerting Failures

Description: - Information Loss of Omission is a security issue that occurs when important information is missing from data processing, reporting, or system communication. This can happen when an application fails to include critical details during logging, auditing, or data transmission. As a result, important events or actions may not be properly recorded or communicated. Attackers may exploit this weakness to hide malicious activities within the system. It may also cause inaccurate system behavior or incomplete data processing. This issue often occurs due to poor system design or insufficient validation. Proper logging, monitoring, and data validation help reduce this risk. **Business Impact:-** Information loss due to omission can lead to incomplete records and inaccurate system reports. Important security events may go unnoticed by administrators. This can allow attackers to perform malicious actions without detection. Organizations may face operational problems due to missing or incorrect information. It can also affect decision-making and system reliability.

Businesses may suffer financial losses if errors or security issues are not properly tracked. Maintaining accurate logging and monitoring is essential for protecting business systems.

The screenshot shows the Common Weakness Enumeration (CWE) website. The header includes the CWE logo, the text "Common Weakness Enumeration" and "A community-developed list of SW & HW weaknesses that can become vulnerabilities". There are also badges for "Top 25" and "HW CWE", and a link "New to CWE? Start here!". The navigation bar includes links for Home, About, Learn, Access Content, Community, and Search. The main content area is titled "CWE-221: Information Loss or Omission". It shows the Weakness ID: 221, Vulnerability Mapping: ALLOWED (with careful review of mapping notes), and Abstraction: Class. There are buttons for "View customized information" with options: Conceptual, Operational, Mapping Friendly, Complete, and Custom. The "Description" section states: "The product does not record, or improperly records, security-relevant information that leads to an incorrect decision or hampers later analysis." The "Relationships" section is expanded, showing "Relevant to the view 'Research Concepts' (View-1000)". It contains a table with columns: Nature, Type, ID, and Name. The table lists several relationships, including ChildOf, ParentOf, and GrandChildOf, with IDs like 554, 222, 223, 224, 256, 296, 297, and 451. The "Modes Of Introduction" section is also expanded, showing a table with columns: Phase and Note. The phases listed are Architecture and Design, and Implementation.

10. VulnerabilityName:- NULL Pointer Dereference

CWE/CVE: CWE-476

OWASP/SANS Category: - A10:2025 Mishandling of Exceptional Conditions **Description:** - NULL Pointer Dereference is a software vulnerability that occurs when a program attempts to use a pointer that has not been assigned a valid memory address. A null pointer points to no location in memory. When the program tries to access or modify data using this pointer, it can cause the application to crash. This issue often occurs due to poor error handling or improper memory

management. Attackers may exploit this vulnerability to cause denial of service by crashing the system. It is common in low-level programming languages such as C and C++. Proper validation and pointer checks can help prevent this vulnerability. **Business Impact:-** NULL pointer dereference can cause applications or systems to crash unexpectedly. This may lead to service interruptions or system downtime. Businesses may experience operational disruptions if critical systems stop functioning. Customers may be unable to access services during such failures. Repeated system crashes can reduce user trust in the application. Organizations may also incur costs for system recovery and maintenance. Proper coding practices and testing help prevent such reliability and security issues.

 **Common Weakness Enumeration**
A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25

Top HW CWE

New to CWE?
Start here!

Home > CWE List > CWE-476: NULL Pointer Dereference (4.19.1)

Home | About | Learn | Access Content | Community | Search

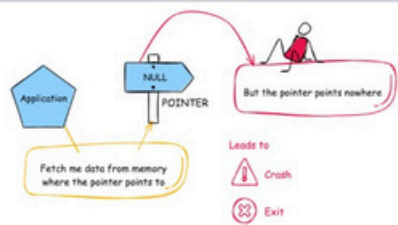
CWE-476: NULL Pointer Dereference

Weakness ID: 476
Vulnerability Mapping: **ALLOWED**
Abstraction: Base

View customized information: Conceptual Operational Mapping Friendly Complete Custom

Description

The product dereferences a pointer that it expects to be valid but is NULL.



Alternate Terms

NPD	Common abbreviation for Null Pointer Dereference
null deref	Common abbreviation for Null Pointer Dereference
NPE	Common abbreviation for Null Pointer Exception

Stage 2

OVERVIEW :-

Nessus is a widely used vulnerability scanner that automates the detection of security weaknesses across networks, systems, and applications. From the screenshots you shared, I understand it as a comprehensive tool from Tenable that performed an informational scan on a remote web host, identifying configuration details like TLS support and missing headers rather than critical exploits.

Core Functionality

Nessus works by launching plugins—pre-built scripts that probe targets for known vulnerabilities, misconfigurations, and service details. In your scan (version 10.8.1), it ran checks in parallel on one host, fingerprinting the OS, detecting a Lua-based web service on a port, and noting informational items like supported TLS protocols (via plugin 10746) and additional DNS names (plugin 46180).

It categorizes findings by severity: your results show low or info-level issues, such as HSTS missing (plugin 108098), which leaves HTTPS traffic vulnerable to interception, and TLS Next Protocol Negotiation enabled, advertising protocols like HTTP/2 that could signal outdated setups.

Key Features Observed

- **Service Detection:** Identifies web servers via banners or errors, as in your "NET Service Detection" output stating "A web is running on this port through Lua!" This helps map attack surfaces.
- **Protocol Checks:** Enumerates TLS extensions (plugin 62564) and supported versions, flagging deprecated ones for remediation.
- **Fingerprinting:** Gathers OS and host details without exploits, using methods like PCRE probes for high-confidence matches.
- **Compliance Auditing:** Spots absences like HSTS headers or restricted methods disclosure, providing vendor fix guidance.

Scan Workflow Insight

Your scan followed a standard flow: host discovery, port/service enum, vulnerability plugins, and reporting. Outputs include plugin IDs (e.g., 13524 for version leaks), risk factors (none high here), and remediation notes like adding Strict-Transport-Security headers. **Reports export to formats like PDF/HTML for sharing security posture overviews.**

Target website :- public-firing-range.appspot.com

Target ip address:- 142.251.31.153

List of vulnerability :-

S.No	Vulnerability Name	Severity	
1 2	Cross-Site Scripting (XSS)	High	11219
3 4	Angular Template Injection	High	11213
5 6	Insecure Third-Party Script Import	Medium	11213
7 8	Mixed Content Detected	Medium	98056
9	Cross-Origin Resource Sharing (CORS) Misconfiguration	Medium	104743
10	Clickjacking Vulnerability	Medium	98060
	Reverse Clickjacking	Medium	11216
	Cookie Without HttpOnly Flag	Medium	10010
	Remote Script Inclusion	Medium	11219
	Flash Object Injection	Low	65821

REPORT:-

Vulnerability Name: Cross-Site Scripting (XSS)

Severity

High

Plugin

11219

Port

443

Description

Cross-Site Scripting (XSS) occurs when a web application accepts untrusted user input and includes it in the output without proper validation or encoding. Attackers may inject malicious JavaScript code that executes in the victim's browser when the vulnerable page is loaded.

Solution

Developers should implement proper input validation and output encoding before displaying user-supplied data. Additionally, security headers such as Content Security Policy (CSP) can be implemented to reduce the risk of XSS attacks.

Business Impact

Attackers may exploit XSS vulnerabilities to steal session cookies, hijack user sessions, redirect users to malicious websites, or perform unauthorized actions on behalf of the victim.

Vulnerability Name: Content Security Policy (CSP) Missing

Severity

Medium

Plugin

98059

Port

443

Description

Content Security Policy (CSP) is a browser security feature that helps prevent attacks such as Cross-Site Scripting and data injection. The absence of a CSP header means that browsers will not restrict the sources from which scripts and other resources can be loaded.

Solution

Administrators should configure the server to include a Content Security Policy header.

Example:

Content-Security-Policy: default-src 'self'

Business Impact

Without CSP protection, attackers may inject malicious scripts into the application, which could compromise user data or allow unauthorized actions within the application.

Vulnerability Name: Cookie Without HttpOnly Flag

Severity

Medium

Plugin

10010

Port

443

Description

Cookies that are not marked with the HttpOnly flag may be accessible through client-side scripts. This increases the risk of cookie theft through Cross-Site Scripting attacks.

Solution

Sensitive cookies should be configured with the following security flags:

Set-Cookie: sessionid=value; HttpOnly; Secure; SameSite=Strict

Business Impact

If attackers obtain session cookies, they may hijack user sessions and gain unauthorized access to user accounts.

Vulnerability Name: Mixed Content Detected

Severity

Medium

Plugin

98056

Port

443

Description

Mixed content occurs when a secure HTTPS page loads resources such as scripts or images over an insecure HTTP connection.

Solution

All resources should be loaded using HTTPS to ensure secure communication between the client and server.

Business Impact

Attackers may intercept or modify insecure HTTP resources, potentially injecting malicious code into the webpage.

Vulnerability Name: Cross-Origin Resource Sharing (CORS) Misconfiguration

Severity

Medium

Plugin

104743

Port

443

Description

CORS misconfiguration occurs when a web server allows unauthorized domains to access sensitive resources through cross-origin requests.

Solution

Administrators should restrict CORS access only to trusted domains.

Example:

Access-Control-Allow-Origin: <https://trusted-domain.com>

Business Impact

Improper CORS configuration may allow malicious websites to access sensitive information from the application.

Vulnerability Name: Clickjacking Vulnerability

Severity

Medium

Plugin

98060

Port

443

Description

Clickjacking occurs when a malicious website embeds the vulnerable site within an invisible iframe and tricks users into clicking hidden elements.

Solution

Implement frame protection headers such as:

X-Frame-Options: DENY

Content-Security-Policy: frame-ancestors 'none'

Business Impact

Attackers may manipulate users into performing unintended actions such as approving transactions or changing account settings.

Vulnerability Name: HTTP TRACE / TRACK Methods Allowed

Severity

Medium

Plugin

42873

Port

443

Description

The HTTP TRACE or TRACK methods allow clients to view the exact request received by the server. These methods may expose sensitive information and facilitate cross-site tracing attacks.

Solution

Disable unnecessary HTTP methods such as TRACE and TRACK in the web server configuration.

Business Impact

Attackers may exploit this feature to capture authentication headers or session cookies.

Vulnerability Name: Web Server Information Disclosure

Severity

Info

Plugin

22964

Port

443

Description

The web server discloses information about its software or configuration through HTTP headers or error messages.

Solution

Administrators should remove or limit server banners and version information in HTTP responses.

Business Impact

Attackers may use this information to identify potential vulnerabilities in the server software.

Vulnerability Name: TLS Protocols Supported

Severity

Info

Plugin

10746

Port

443

Description

This plugin identifies the TLS protocols supported by the server during encrypted communication.

Solution

Only secure protocols such as TLS 1.2 and TLS 1.3 should be enabled, while older protocols such as SSL and TLS 1.0 should be disabled.

Business Impact

Using outdated TLS protocols may expose encrypted communications to cryptographic attacks.

Vulnerability Name: Nessus Scan Information

Severity

Info

Plugin

11936

Port

443

Description

This plugin provides information about the vulnerability scan itself, including the scanning tool version and the number of hosts scanned.

Solution

No remediation is required because this finding only provides metadata about the scanning process.

Business Impact

This information does not pose a direct security risk but is useful for documentation and auditing purposes.

Stage – 3

Report

Title of the Project: -

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

1. Centralized Identity Management

Centralized Identity Management is a key component in implementing Zero Trust Network Access (ZTNA) using Zscaler. In traditional network security models, access decisions are often based on the user's network location, such as whether they are inside or outside the corporate network. However, in a Zero Trust model, access decisions are based primarily on verified user identity. Zscaler integrates with enterprise identity providers such as Active Directory, Azure Active Directory, Okta, and other Identity and Access Management (IAM) systems to create a centralized identity management framework.

Through centralized identity management, organizations can maintain a single source of truth for user identities, roles, and permissions. All user authentication and authorization processes are handled through the centralized identity system, ensuring consistency across different applications and services. This simplifies administrative tasks, as IT teams no longer need to manage multiple user accounts or access policies across different systems. Instead, identity information is synchronized and managed in one location.

Another major benefit is improved security. Since ZTNA relies on identity-based access rather than network access, every request to access an application is verified based on the user's identity, device posture, and security policies. This ensures that only authenticated and authorized users can access specific resources. Centralized identity management also allows organizations to quickly update access privileges when employees change roles or leave the organization. By implementing centralized identity management within Zscaler ZTNA, organizations can achieve stronger security, simplified administration, and more efficient access control.

2. Single Sign-On (SSO) Authentication

Single Sign-On (SSO) authentication plays an important role in improving both security and user experience in a ZTNA environment. SSO allows users to authenticate once and gain access to multiple applications without repeatedly entering their credentials. When integrated with Zscaler ZTNA, SSO provides a seamless and secure method for accessing enterprise applications while maintaining strong authentication controls.

In a traditional environment, users often have to remember and manage multiple usernames and passwords for different applications. This can lead to password fatigue, where users reuse weak passwords or write them down, increasing security risks. SSO eliminates this problem by allowing users to log in once through a trusted identity provider and gain access to authorized applications. Once authenticated, Zscaler verifies the user's identity and grants access according to predefined security policies.

From a security perspective, SSO reduces the number of authentication points and centralizes authentication processes. Administrators can enforce strong authentication policies through the identity provider, such as password complexity rules or multi-factor authentication. If a user's account needs to be disabled or modified, administrators can update the identity provider, and the changes automatically apply to all connected applications. This simplifies identity management while maintaining high security standards.

Overall, SSO improves productivity by allowing users to quickly access the resources they need while ensuring that access remains secure and centrally controlled.

3. Enhanced Security and Access Control

One of the primary objectives of implementing ZTNA using Zscaler is to enhance security and strengthen access control mechanisms. Traditional network security approaches typically grant users access to an entire internal network once they successfully authenticate through a VPN. This model increases the risk of lateral movement within the network if an attacker compromises a user account. ZTNA eliminates this risk by providing application-level access rather than network-level access.

With Zscaler ZTNA, users are granted access only to specific applications they are authorized to use. This approach follows the principle of least privilege, which ensures that users receive only the minimum level of access necessary to perform their tasks. Access decisions are based on multiple factors, including user identity, device health, location, and security policies defined by administrators.

Additionally, Zscaler continuously evaluates access requests in real time. If any suspicious activity or security risk is detected, the system can automatically block or restrict access. This dynamic access control mechanism significantly reduces the attack surface and prevents unauthorized access to sensitive resources.

By restricting access to individual applications and continuously verifying security conditions, organizations can protect critical systems from cyber threats such as credential theft, insider attacks, and unauthorized access. This approach provides a stronger and more flexible security framework compared to traditional network security models.

4. Automated User Provisioning and Deprovisioning

Automated user provisioning and deprovisioning are essential features for efficiently managing user access in modern organizations. In a ZTNA environment implemented with Zscaler, automation ensures that users receive the appropriate level of access based on their roles within the organization. This process is typically integrated with identity management systems and directory services.

User provisioning refers to the process of granting access to applications and resources when a new employee joins the organization or when an existing employee requires additional access. Instead of manually configuring access for each user, administrators can define role-based policies that automatically assign permissions based on job roles or departments. This reduces administrative workload and ensures consistent access control policies across the organization.

Deprovisioning is equally important for maintaining security. When an employee leaves the organization or changes roles, their access to certain applications must be immediately revoked or modified. Automated deprovisioning ensures that these changes occur instantly through the identity management system. This prevents former employees or unauthorized users from retaining access to sensitive systems and data.

Automation also reduces the risk of human error, which is a common cause of security vulnerabilities in manual access management processes. By implementing automated provisioning and deprovisioning within Zscaler ZTNA, organizations can maintain accurate access controls while improving operational efficiency and security.

5. Multi-Factor Authentication (MFA) Support

Multi-Factor Authentication (MFA) is a critical security feature that strengthens the authentication process by requiring users to provide multiple forms of verification. In addition to entering a password, users must confirm their identity using an additional factor such as a mobile authentication app, a one-time password, biometric verification, or a hardware security token.

When Zscaler ZTNA is integrated with identity providers that support MFA, organizations can significantly enhance their security posture. Even if an attacker obtains a user's password through phishing or other attacks, they would still need the additional authentication factor to gain access. This greatly reduces the likelihood of unauthorized access to enterprise applications.

MFA is particularly important in modern work environments where employees access corporate applications from various locations and devices. By requiring multiple forms of authentication, organizations can ensure that only legitimate users can access sensitive resources. Administrators can also configure MFA policies based on risk factors, such as requiring additional verification when users attempt to access applications from unfamiliar devices or locations.

Overall, MFA adds an essential layer of security to ZTNA deployments and helps organizations protect their systems from credential-based attacks.

6. Improved Monitoring, Logging, and Compliance

Effective monitoring and logging are essential for maintaining visibility into user activities and ensuring that security policies are being properly enforced. Zscaler ZTNA provides detailed monitoring and logging capabilities that allow administrators to track user access to applications in real time. These logs include information such as user identity, device details, access location, and the specific applications accessed.

This visibility enables security teams to detect suspicious behavior and respond quickly to potential security threats. For example, if a user attempts to access an application from an unusual location or device, administrators can investigate the activity and take appropriate action. Continuous monitoring also helps organizations identify potential vulnerabilities and improve their overall security posture.

In addition to improving security, logging and monitoring support regulatory compliance requirements. Many organizations must comply with industry regulations and standards that require detailed records of user access and security events. Zscaler's reporting and logging capabilities provide the necessary documentation to demonstrate compliance with these requirements.

By providing comprehensive monitoring, logging, and reporting features, Zscaler ZTNA helps organizations maintain strong security governance, quickly detect threats, and ensure compliance with regulatory standards.